Sample document TV-ENG-2024-SAMPLE-002

Engineering Runbook v2

TechVision Platform Engineering — Operations Runbook

Document	Engineering Runbook v2
Version	Runbook v2.1 (Sample)
Reference	Sample document TV-ENG-2024-SAMPLE-002
Issued	September 2026
Owner	Omen United Insurance / TechVision Solutions (fictional)
Classification	Sample — demo knowledge base content

IMPORTANT NOTICE — This is a fictional sample document generated for the Clara demo environment. Every name, number, figure, address and procedure in this document is invented. Nothing here is real insurance, legal, medical or engineering advice, and no real entity, product or service is described. The document exists so that demo assistants can retrieve and cite realistic structured content.

1. Purpose and Scope

This is a SAMPLE runbook created for demonstration purposes. TechVision Solutions is a fictional company, and every command, hostname, port, threshold and escalation contact in this runbook is invented. The runbook exists so that assistants in the TechVision demo knowledge base can answer operational questions about environment setup, deployment, rollback, backups, monitoring and on-call escalation.

The runbook covers the platform's primary services: the API tier, the worker tier that executes pipeline runs, the scheduler service, and the shared Postgres and Redis dependencies. It is written for the on-call engineer holding the platform pager, and assumes familiarity with the fictional internal tooling named in each section. Change management requires that every procedure here be executed exactly as written, with deviations recorded in the incident channel.

2. Development Environment Setup

How do I set up a new development environment? The supported local stack runs on Node.js 22 with the fictional package manager, a local Postgres 16 on port 5433, and Redis 7 on 6380. The bootstrap sequence below brings a laptop from zero to a running platform in roughly 20 minutes.

- Install Node.js 22.x and verify with node --version; the fictional toolchain refuses other majors.
- Clone the platform repository and copy .env.example to .env — the example file contains working localhost defaults.
- Run the database bootstrap: ./scripts/dev-db-up.sh starts Postgres 16 on 5433 and Redis 7 on 6380 in detached mode.
- Install dependencies with the frozen lockfile: pnpm install --frozen-lockfile.
- Apply migrations: pnpm db:migrate — idempotent, safe on an existing volume.
- Seed demo data: pnpm db:seed — creates the fictional sample orgs and pipelines.
- Start the stack: pnpm dev — API on localhost:4000, worker and scheduler in watch mode.

The verification checklist after bootstrap: hit http://localhost:4000/healthz and expect 200 OK; run pnpm test:smoke for the 12-check smoke suite; and confirm the worker registers by checking the workers table has a heartbeat newer than 60 seconds. Common setup failures are a stale volume from an old branch — fixed with pnpm db:reset — and a port conflict on 5433 from a previous detached container.

3. CI/CD Pipeline Overview

Every merge to main triggers the fictional CI pipeline: lint and typecheck in parallel, the unit suite next, then an integration suite that boots the real Postgres and Redis service containers, and finally a container build that produces the api and worker images tagged with the commit SHA. Merges to release branches additionally run the deployment rehearsal, which applies migrations against a throwaway copy of the production snapshot.

Deployments are promoted through three environments: staging receives every main build automatically; canary receives builds promoted by the release manager; production receives builds only after canary metrics pass their gates for 30 minutes — error rate under 0.5%, p99 latency within 10% of baseline, and worker queue depth back to steady state. A failed gate halts promotion and pages the release manager, never auto-rolling back silently.

4. Deploying to Production

How do I deploy to production? Production deployments run through the fictional deploy CLI and follow the sequence below. Total downtime is zero: the API tier is behind a rolling load balancer, and the worker tier drains in-flight runs before replacing each pod.

- Step 1 — Announce the deploy in #platform-release with the release candidate SHA and the changelog link.
- Step 2 — Confirm canary gates passed: deployctl status canary must show all gates green for the RC.
- Step 3 — Run the preflight: deployctl preflight prod — verifies migrations are dry-run clean, disk headroom, and replication lag under 5 seconds.
- Step 4 — Apply migrations: deployctl migrate prod — migrations are expand-only and backward compatible by convention; contract-breaking changes ship in two phases.
- Step 5 — Roll the API tier: deployctl rollout prod api — 10%, then 50%, then 100%, with automatic halt on error-budget burn.
- Step 6 — Roll the worker tier: deployctl rollout prod worker — drains each pod for up to 15 minutes before replacement.
- Step 7 — Verify: deployctl verify prod — checks healthz on every pod, synthetic read/write probes, and scheduler heartbeat.
- Step 8 — Close the release in #platform-release with the verify summary and mark the changelog entry shipped.

Deployment windows are business hours on Tuesdays and Thursdays; emergency deploys outside windows require a second approver through the break-glass workflow, which logs to the audit trail automatically. Never apply a migration that the preflight flags as contracting without the two-phase plan attached.

5. Rollback Procedure

How to roll back a failed deployment? Rollback is the first remedy whenever production error rate exceeds 1% for 5 minutes or p99 latency doubles after a rollout. Roll forward only when the fix is already built and verified; otherwise roll back.

- Step 1 — Freeze the rollout: deployctl rollout prod api --pause stops further percentage increases immediately.
- Step 2 — Roll back the API tier: deployctl rollback prod api --to <previous-SHA> — redeploys the previous image; takes about 3 minutes.
- Step 3 — Roll back the worker tier: deployctl rollback prod worker --to <previous-SHA>.
- Step 4 — Database: because migrations are expand-only, no down-migration is performed. The rollback runbook keeps the expanded columns and the old code simply ignores them.
- Step 5 — Verify recovery: deployctl verify prod plus watch the error rate and queue depth dashboards for 15 minutes.
- Step 6 — Post-incident: file the rollback note in the incident record; the platform retrospective reviews whether the release gate missed the defect class.

If the defect is in a migration's data transformation rather than the code, rollback alone does not heal data: engage the data-recovery procedure in the incident runbook, restore affected rows from the nearest snapshot, and only then roll forward with the corrected migration. The two-phase migration convention exists precisely to keep this window rare.

6. Database Backup and Restore

What are the database backup procedures? Automated backups run nightly at 02:00 UTC: a full physical snapshot of the Postgres data volume, shipped to the fictional object storage bucket with 30-day retention, plus continuous write-ahead-log archiving that exposes a 5-minute recovery point objective. A weekly logical dump (pg_dump, compressed) is retained 90 days for cross-region restores and partial table recovery.

- Verify the latest snapshot: backupctl status — shows last success, size, and integrity check result.
- Restore to a scratch instance: backupctl restore --snapshot latest --to scratch-<ticket> — provisions an isolated clone in about 12 minutes.
- Point-in-time recovery: backupctl restore --pitr '2026-09-17 09:35' --to scratch-<ticket> — replays WAL to the minute.
- Partial table recovery: restore the weekly logical dump, extract the table, and load it with the standard upsert script.

The quarterly restore drill is mandatory: each rotation, the on-call restores last night's snapshot to a scratch instance, runs the smoke suite against it, and records the measured restore time in the runbook's drill log. The recovery time objective is 60 minutes for a full region loss, using the cross-region replica and the weekly logical dump; the replica can be promoted with replicactl promote --force, which pages the secondary on-call for confirmation.

7. Monitoring and Alerting

What monitoring tools do we use? The fictional observability stack is: Prometheus for metrics with 15-second scrape intervals; Grafana for dashboards, with the golden-signals board open during incidents; Sentry for API error tracking with release tagging; and the ELK-style log cluster for structured logs, searched through the fictional Kibana front-end. Synthetic probes run from three external regions every 30 seconds against healthz and a read/write pair.

Alerts route through Alertmanager to the on-call rotation, with severity levels matching the incident levels in Section 9. Burn-rate alerting protects the error budget: a 2% burn in 1 hour or a 5% burn in 6 hours pages immediately. Queue depth on the worker tier alerts at 1,000 pending runs for 10 minutes; replication lag alerts at 15 seconds; certificate expiry alerts at 21 days.

Dashboard conventions: every service panel names its owner team, every alert links to its runbook section here, and every dashboard carries a link to the on-call calendar. If an alert fires without a linked runbook section, the on-call files a documentation bug — that is a defect in the runbook, not noise to be silenced.

8. On-Call Escalation Process

What is the on-call escalation process? Primary on-call holds the pager for the platform rotation, one week at a time, handing over at 10:00 UTC on Mondays with a written handover note. Unacknowledged pages escalate after 5 minutes to the secondary, after another 5 minutes to the engineering manager, and after a further 10 minutes to the fictional VP of Engineering. Break-glass access to production escalates in parallel with the page, requiring the secondary's approval for any change outside the procedures in this runbook.

Timeline	Escalation target
T+0	Primary on-call paged (Sev-1/Sev-2)
T+5 min unack	Secondary on-call paged
T+10 min unack	Engineering manager paged
T+20 min unack	VP Engineering paged; incident bridge opened

Every Sev-1 incident opens a bridge call and an incident channel automatically from the pager payload. The incident commander role separates coordination from investigation: the commander runs the timeline, calls for rollbacks, and is the only voice authorising production changes. Customer communication drafts live in the incident template library and are posted by the communications lead, not by whoever happens to be typing.

9. Incident Severity Levels

Severity	Definition (fictional)	Response target
Sev-1	Production down or data loss risk; >50% of requests failing	Acknowledge 5 min, bridge immediately
Sev-2	Major feature degraded; error rate 1-50%; no data risk	Acknowledge 15 min, update hourly
Sev-3	Minor degradation or cosmetic defect with workaround	Acknowledge 1 business day
Sev-4	Bug in non-production path or documentation gap	Next sprint triage

Severity is set by the incident commander at declaration and re-evaluated every 30 minutes; a Sev-2 that breaches the 50% error threshold is upgraded without waiting for the scheduled review. Post-incident reviews are blameless, scheduled within 5 business days for Sev-1 and Sev-2, and every action item gets an owner and a due date tracked to closure in the engineering review.

10. Glossary and Closing

Term	Meaning (fictional)
RC	Release candidate — a commit SHA promoted through canary to production.
RPO / RTO	Recovery point / recovery time objective for the backup regime.
Burn rate	Speed of error-budget consumption used for alerting.
Expand-only migration	A schema change that old code can safely ignore.
Break-glass	The emergency approval path for out-of-procedure changes.

TechVision Solutions is a fictional company; this runbook is a demonstration artefact for the TechVision knowledge base. All commands, hosts, thresholds and escalation targets are invented, and no real infrastructure is described.